endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 1

PROGRAMA

1. Introducere în limbajul Java
2. Clase și obiecte. Mostenirea claselor. Polimorfism
3. Tablouri. Șiruri de caractere
4. Clase abstracte. Interfețe
5. Excepții
6. Fluxuri de intrare/ieșire
7. Colecții de date
8. Expresii lambda si Streams API
9. Design patterns
10. Fire de executie
11. Sockets
12. Baze de date (JDBC)

endava

EVALUARE

1. Curs 50%
   1. Examen scris (verificare) in ultima saptamana din semestru
2. Laborator 50%
   1. Prezenta si activitate 25%
   2. Proiect 25%

Note:

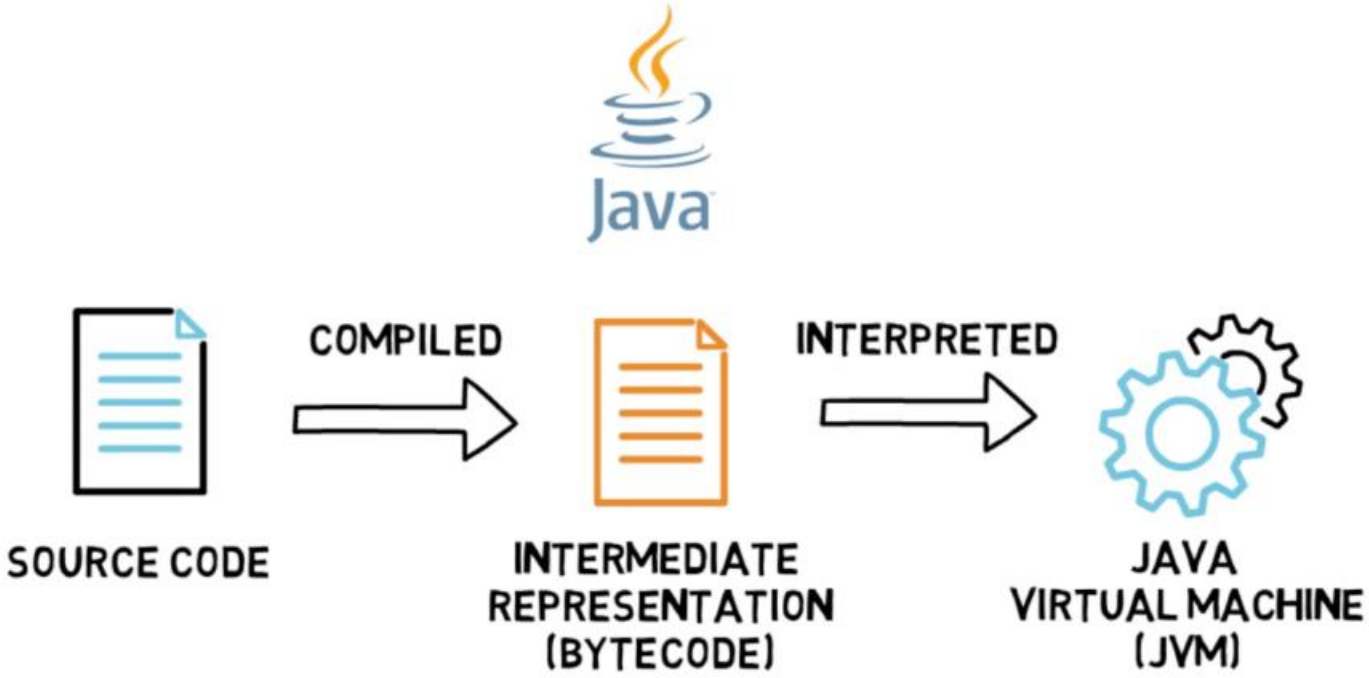
- Nota din examen trebuie sa fie minim 5 fara rotunjire pentru a promova
- Nota din laborator trebuie sa fie minim 5 fara rotunjire pentru a putea sustine examenul
- Se va face o singura rotunjire, dupa insumarea notei din laborator cu nota din examen

AGENDA CURS 1

1. Hey, Java
2. Hello world!
3. Tipuri de date
4. Operatori
5. Instructiuni decizionale
6. Instructiuni repetitive

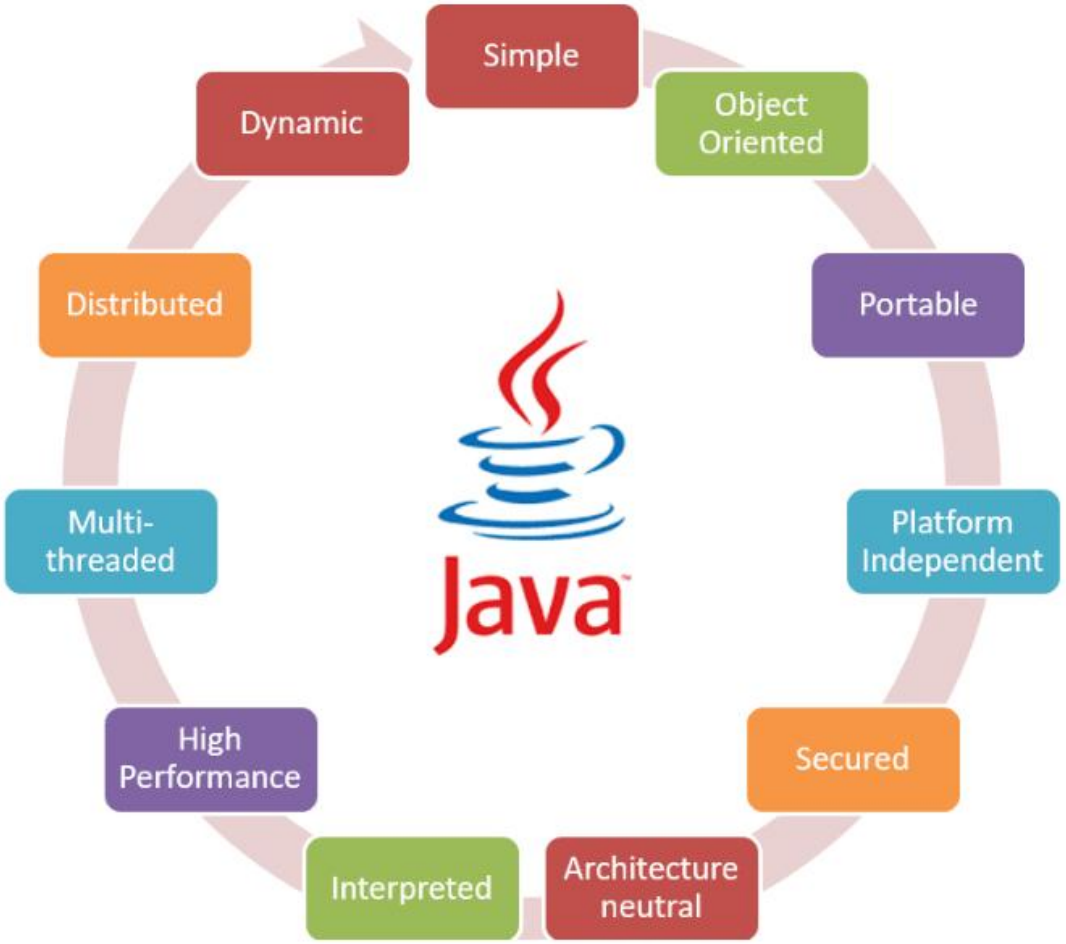
endava

HEY, JAVA



endava

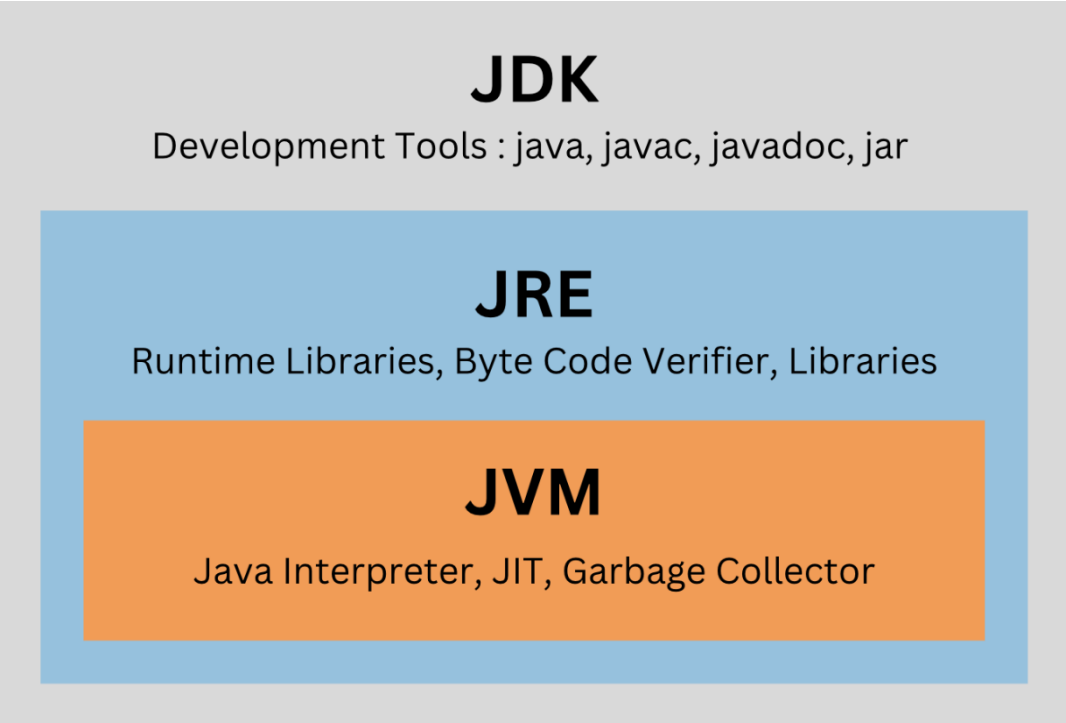
HEY, JAVA



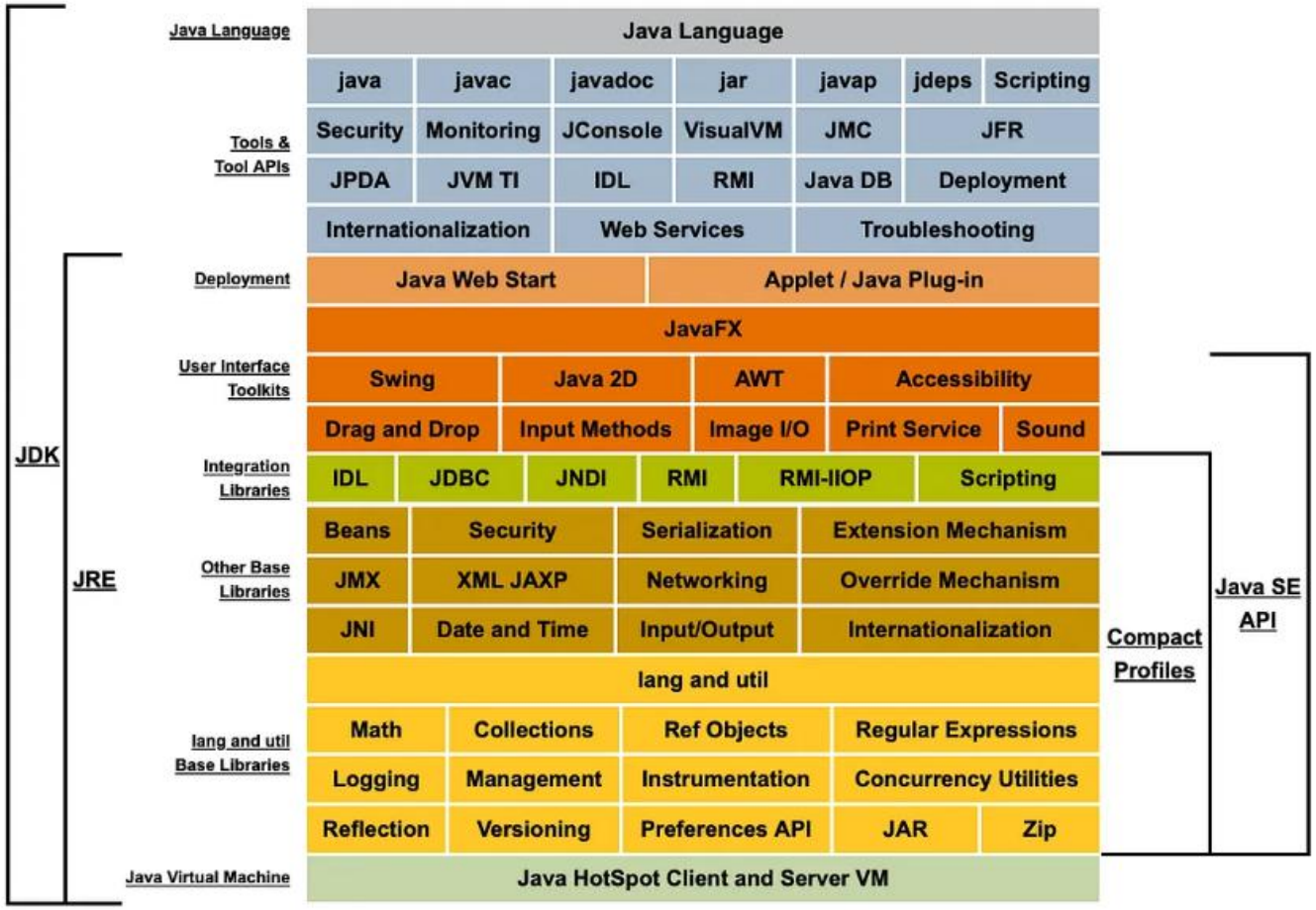
endava

HEY, JAVA

- **JDK** – Java Develoment Kit
  - folosit pentru a dezvolta si rula aplicatii Java
  - javac – Java compiler
  - jar – Java archive
- **JRE** – Java Runtime Environment
  - folosit doar pentru a rula aplicatii Java
- **JVM** – Java Virtual Machine
  - Traduce byte code in limbaj masina pentru platforma pe care se va rula aplicatia Java



HEY, JAVA

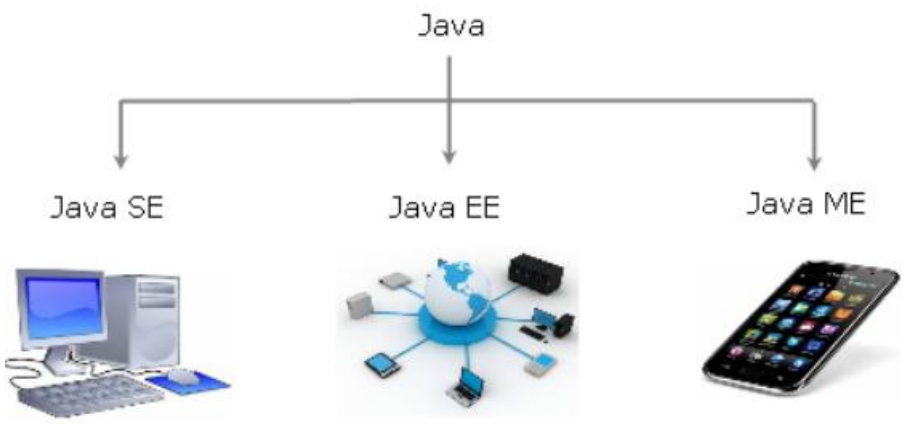


https://docs.oracle.com/javase/8/docs/index.html

endava

HEY, JAVA

- Java platforms:
  - **Java SE** – Java Standard Edition
    - aplicatii desktop
    - contine elementele fundamentale de limbaj si APIs de baza
    - tehnologii precum JDBC
  - **Java ME** – Java Micro Edition
    - aplicatii pe small devices, cum ar fi telefoane sau automate
  - **Java EE** – Java Enterprise Edition
    - aplicatii enterprise, Web, distribuite, cu logica de business complexa
    - tehnologii precum JSP, JSF, Servlet, JPA, DI



HELLO WORLD!

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```


endava

TIPURI DE DATE

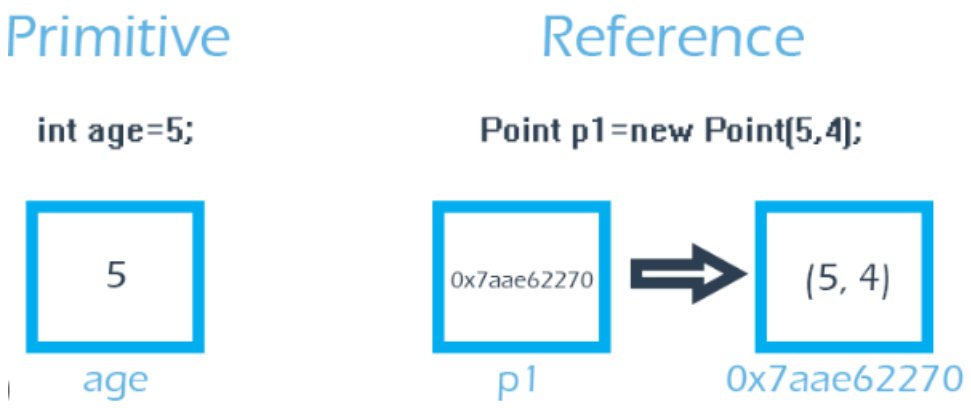
1. Tipuri primitive

DATA TYPES	SIZE	DEFAULT	EXPLAINATION
boolean	1 byte/ 8bits	false	Stores true or false values
byte	1 byte/ 8bits	0	Stores whole numbers from -128 to 127
short	2 bytes/ 16bits	0	Stores whole numbers from -32,768 to 32,767
int	4 bytes/ 32bits	0	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes/ 64bits	0L	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes/ 32bits	0.0f	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes/ 64bits	0.0d	Stores fractional numbers. Sufficient for storing 15 decimal digits
char	2 bytes/ 16bits	'\u0000'	Stores a single character/letter or ASCII values

endava

TIPURI DE DATE

2. Tipuri referinte (non-primitive)



OPERATORI

Precedence	Operator	Type	Associativity
15	() [] .	Parentheses Array subscript Member selection	Left to Right
14	++ --	Unary post-increment Unary post-decrement	Left to Right
13	++ -- + - ! ~ (*type*)	Unary pre-increment Unary pre-decrement Unary plus Unary minus Unary logical negation Unary bitwise complement Unary type cast	Right to left
12	* / %	Multiplication Division Modulus	Left to right
11	+ -	Addition Subtraction	Left to right
10	<< >> >>>	Bitwise left shift Bitwise right shift with sign extension Bitwise right shift with zero extension	Left to right
9	< <= > >= instanceof	Relational less than Relational less than or equal Relational greater than Relational greater than or equal Type comparison (objects only)	Left to right
8	== !=	Relational is equal to Relational is not equal to	Left to right
7	&	Bitwise AND	Left to right
6	^	Bitwise exclusive OR	Left to right
5	|	Bitwise inclusive OR	Left to right
4	&&	Logical AND	Left to right
3	||	Logical OR	Left to right
2	? :	Ternary conditional	Right to left
1	= += -= *= /= %=	Assignment Addition assignment Subtraction assignment Multiplication assignment Division assignment Modulus assignment	Right to left

Larger number means higher precedence.

endava

INSTRUCTIUNI DECIZIONALE - IF

- Forma de baza:

```
if (condition) {
    // Code executes if condition is true
}
```

```
int number = 10;
if (number > 0) {
    System.out.println("Positive number");
} else if (number < 0) {
    System.out.println("Negative number");
} else {
    System.out.println("Zero");
}
```

```
if (condition) {
    // Executes if condition is true
} else if (anotherCondition) {
    // Executes if the first condition is false, but this one is true
} else {
    // Executes if none of the above conditions are true
}
```


endava

INSTRUCTIUNI DECIZIONALE - IF

- Exemplu forma nested:

```
int num = 15;
if (num > 0) {
    if (num % 2 == 0) {
        System.out.println("Positive even number");
    } else {
        System.out.println("Positive odd number");
    }
}
```


endava

INSTRUCTIUNI DECIZIONALE - SWITCH

- Pentru versiuni < Java 12:
  - Poate fi folosit cu byte, short, char, int, String, enum, si clasele wrapper
  - Este nevoie de ***break*** pentru a evita executia mai multor ramuri

```
switch (expression) {
    case value1:
        // Code to execute if expression == value1
        break;
    case value2:
        // Code to execute if expression == value2
        break;
    default:
        // Code executes if none of the cases match
}
```

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Invalid day");
}
```

INSTRUCTIUNI DECIZIONALE - SWITCH

- Pentru versiuni >= Java 12:
  - nu mai este nevoie de ***break*** pentru a evita executia mai multor ramuri
  - poate fi folosit arrow syntax (**->**) pentru a avea expresii concise
  - poate fi folosit ***return***

```
switch (expression) {
    case value1 -> statement1;
    case value2 -> statement2;
    default -> defaultStatement;
}
```

```
int day = 3;
switch (day) {
    case 1 -> System.out.println("Monday");
    case 2 -> System.out.println("Tuesday");
    case 3 -> System.out.println("Wednesday");
    default -> System.out.println("Invalid day");
}
```

INSTRUCTIUNI DECIZIONALE - SWITCH

- Pentru versiuni >= Java 14:
  - poate fi folosit ca o expresie (care sa returneze o valoare)
  - poate testa mai multe valori pe o ramura

```
int month = 4;
String season = switch (month) {
    case 12, 1, 2 -> "Winter";
    case 3, 4, 5 -> "Spring";
    case 6, 7, 8 -> "Summer";
    case 9, 10, 11 -> "Autumn";
    default -> "Invalid month";
};
System.out.println("Season: " + season);
```

INSTRUCTIUNI DECIZIONALE - SWITCH

- Pentru versiuni >= Java 14:
  - poate avea mai multe instructiuni pe o ramura; se foloseste ***yield*** pentru a returna o valoare (in loc de ***return***)

```
int num = 2;
String result = switch (num) {
    case 1 -> "One";
    case 2 -> {
        System.out.println("Processing number 2...");
        yield "Two"; // Return a value
    }
    default -> "Unknown";
};
System.out.println(result);
```

INSTRUCTIUNI DECIZIONALE – TERNARY OPERATOR

- Prescurtare a structurii ***if:***
  - returneaza o valoare pe baza unei conditii
  - este mai rapid pentru conditii simple

```
int num = 10;
String result = (num > 0) ? "Positive" : "Non-positive";
System.out.println(result);
```


endava

INSTRUCTIUNI REPETITIVE - WHILE

- Forma de baza:
  - daca conditia este false la primul pas, nu se executa nicio instructiune din while
  - daca conditia este true la fiecare pas, se va executa la infinit

```java
while (condition) {
    // Code to execute repeatedly
}
```

```java
int count = 1;
while (count <= 5) {
    System.out.println("Count: " + count);
    count++;  // Increment to avoid infinite loop
}
```

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```


endava

INSTRUCTIUNI REPETITIVE - WHILE

- Forma de baza:
  - se poate forta oprirea cu ***break***

```
int num = 1;
while (num <= 10) {
    if (num == 5) {
        break;  // Stops loop at 5
    }
    System.out.println("Num: " + num);
    num++;
}
```

```
Num: 1
Num: 2
Num: 3
Num: 4
```


endava

INSTRUCTIUNI REPETITIVE - WHILE

- Forma de baza:
  - se poate forta trecerea la pasul urmator cu ***continue***

```
int num = 0;
while (num < 5) {
    num++;
    if (num == 3) {
        continue;  // Skips 3
    }
    System.out.println("Num: " + num);
}
```

```
Num: 1
Num: 2
Num: 4
Num: 5
```


endava

INSTRUCTIUNI REPETITIVE – DO-WHILE

- Forma de baza:
  - similar cu ***while***, singura diferenta este ca evaluarea conditiei se face la sfarsit

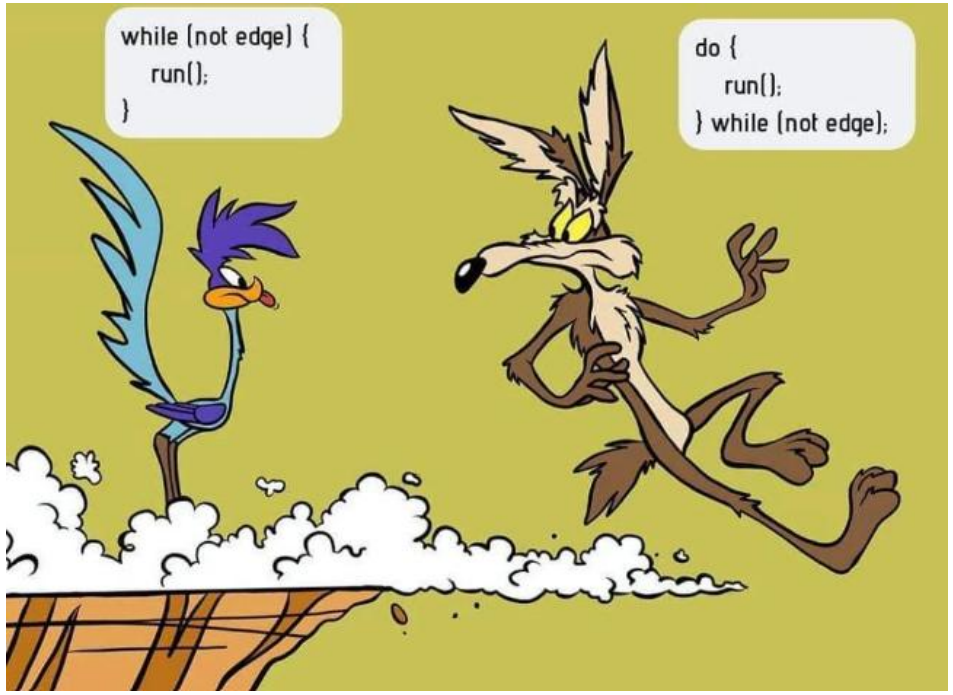
```
do {
    // Code to execute at least once
} while (condition);
```

```
int num = 1;
do {
    System.out.println("Number: " + num);
    num++;
} while (num <= 5);
```

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```


endava

INSTRUCTIUNI REPETITIVE



Caracteristică	`while` Loop	`do-while` Loop
Verificarea condiției	Înainte de execuție	După execuție
Executare garantată	Nu, se poate să nu ruleze deloc	Da, rulează cel puțin o dată
Caz de utilizare	Când vrem să verificăm condiția înainte	Când vrem să executăm codul cel puțin o dată

INSTRUCTIUNI REPETITIVE – FOR

- Forma de baza:
  - partea de initializare se executa o singura data, la primul pas
  - conditia se evalueaza inainte de fiecare pas
  - partea de update face modificari pe variabila implicata in conditie, dupa fiecare pas
  - daca conditia este false la primul pas, nu se executa nicio instructiune din ***for***
  - daca conditia este true la fiecare pas, se va executa la infinit
  - Se poate folosi ***break*** si ***continue***, similar ca pentru ***while***

```java
for (initialization; condition; update) {
    // Code to execute repeatedly
}
```

```java
for (int i = 1; i <= 5; i++) {
    System.out.println("Count: " + i);
}
```

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```


endava

Q&A

endava

THANK YOU

DANIELA SPILCĂ